Large-Scale Job Market Intelligence and Emerging Skill Demand Analysis

An End-to-End Big Data Architecture Using HDFS, Apache Pig, and Apache Hive

Course: CSE412 — Big Data & Large-Scale Computing, IIIT Kottayam

Student Team: N Medhashree (2023BCS0042) · Ashmi S N (2023BCS0196) · Deepanjali Tudu (2023BCS0066)

Repository: `https://github.com/medhashree05/JOB_MARKET_ANALYSIS`

Date of Verified Audit: October 2026

Abstract

Modern online recruitment portals publish tens of thousands of unstructured, heterogeneous job postings weekly. Extracting actionable labour-market intelligence from this corpus requires scalable, fault-tolerant batch architectures capable of handling noisy free text, multi-valued skill lists, inconsistent geographic spellings, and non-standard salary disclosures. In this project, an end-to-end Big Data processing pipeline is designed, implemented, and empirically verified over a primary dataset of 97,929 Indian job postings.

The pipeline integrates three core Hadoop ecosystem layers: (1) **HDFS Ingestion and Sanitization**, which executes deterministic schema validation and deduplication, retaining 97,679 verified postings; (2) **Apache Pig (Pig Latin on MapReduce)**, which executes parallel tokenization, normalisation, distinct job–skill projection, and multi-dimensional aggregation; and (3) **Apache Hive**, which exposes the aggregated partitions as structured external tables via `OpenCSVSerde` and evaluates analytical HiveQL queries across job roles, skills, hiring hubs, and salary benchmarks. A decoupled **Streamlit presentation layer** provides an interactive dashboard with seven pages and dynamic CSV export functionality.

A rigorous technical audit of the repository resolved prior numerical reporting discrepancies: the variance between 97,679 and 97,715 postings and between 18,320 and 18,310 Bengaluru postings is formally traced to a 36-row newline export fragmentation artifact in unquoted intermediate CSV dumps. Furthermore, because the primary dataset represents an active cross-sectional scrape containing relative recency strings rather than multi-year calendar dates, temporal trend measurement is structurally constrained. To address this limitation, the **Skill Demand Velocity Index (SDVI)** is critically evaluated as a conceptual extension; a mathematical audit demonstrates the limitations of single-scalar quadrant thresholding and establishes an orthogonal two-dimensional classification framework.

1. Introduction

Labour market intelligence provides critical signals for curriculum planning, workforce development, and industrial hiring strategy. However, large-scale recruitment data is notoriously noisy: identical job titles appear with arbitrary capitalization and parenthetical annotations;

skill requirements are packed into single comma-delimited strings; salary disclosures are sparse, right-skewed, and multi-currency; and posting dates frequently rely on relative timestamps (e.g., "3 Days Ago") without absolute scrape references. Processing such corpora at scale exceeds the capabilities of in-memory single-node spreadsheets and demands a distributed, fault-tolerant Big Data architecture.

This project delivers a complete, reproducible batch processing pipeline tailored for large-scale job market intelligence:

- **Storage Layer (HDFS):** Distributed ingestion, block-level storage, replication, and data validation.
- **Transformation Layer (Apache Pig):** Procedural MapReduce data flows for schema enforcement, string parsing, skill array flattening, and grouped aggregations.
- **Analytical Warehouse (Apache Hive):** Schema-on-read external table structures, `OpenCSVSerde` parsing for text preservation, and HiveQL analytical querying.
- **Exploration Layer (Streamlit & Plotly):** A decoupled, responsive presentation dashboard delivering executive KPIs, geographic drill-downs, and filtered data exploration.

1.1 Project Scope and Empirical Boundaries

A fundamental principle of scientific computing is maintaining absolute fidelity to empirical evidence. The primary empirical pipeline is verified over the **Indian Job Market Dataset 2025** (97,929 raw records). All reported metrics for role demand, skill popularity, geographic hiring distribution, and disclosed compensation are computed directly from verified execution artifacts on Hadoop 3.3.6 and Hive 3.1.3.

The report explicitly delineates between verified empirical implementations and conceptual extensions:

(a) **Verified Empirical Pipeline (Phase I):** Full ingestion, cleaning, MapReduce execution in Pig, Hive table materialization, and interactive dashboard analytics over 97,679 verified postings.

(b) **Conceptual Velocity Framework (Phase II / SDVI):** A theoretical framework designed to measure longitudinal skill momentum across dated postings, accompanied by a rigorous mathematical critique of its quadrant boundary conditions.

2. Literature Survey

2.1 Distributed Batch Computing Foundations

The architectural foundation of large-scale distributed computing was formulated by Dean and Ghemawat [1], who introduced the MapReduce programming paradigm to parallelise computation across commodity clusters with automatic data partitioning, scheduling, and fault recovery. The underlying storage fabric, the Hadoop Distributed File System (HDFS), described by Shvachko et al. [2], provides high-throughput streaming access to large files via block-level distribution across DataNodes coordinated by a centralised NameNode metadata service.

2.2 High-Level Dataflow and Warehousing Abstractions

Writing native MapReduce in Java involves substantial boilerplate and complex join/aggregation management. To abstract this complexity, Olston et al. [3] developed Pig Latin, a procedural dataflow language that compiles high-level transformations (`FILTER`, `FOREACH`, `GROUP`, `COGROUP`, `FLATTEN`) into directed acyclic graphs of MapReduce jobs. Concurrently, Thusoo et al. [4] introduced Apache Hive, establishing the schema-on-read data warehousing model. Hive projects SQL-like structures (HiveQL) over distributed files, supporting metadata separation, partition pruning, and complex serde implementations such as `OpenCSVSerde`. White [5] provides comprehensive architectural guidance on integrating HDFS, Pig, and Hive within production enterprise clusters.

2.3 Skill Extraction and Labour Market Analytics

Analysing online recruitment advertisements has emerged as a vital discipline in computational economics. Khaouja et al. [6] surveyed text mining and natural language processing approaches for skill extraction, noting that while machine-learning taggers offer high specificity, distributed rule-based tokenisation remains the gold standard for high-throughput batch pipelines. Djumalieva and Sleeman [7] demonstrated that data-driven skill taxonomies derived from online vacancy text accurately track emergent industry requirements long before formal occupation classification systems (such as O*NET or SOC) are updated.

2.4 Momentum Metrics and Percentage Distortion

In longitudinal skill analysis, ranking by raw frequency introduces *incumbent inertia*: foundational skills such as SQL and Python consistently dominate total counts regardless of current market acceleration. Conversely, naive percentage growth suffers from severe *sparsity distortion*: a tool expanding from 1 to 3 postings exhibits +200% growth, vastly overshadowing an enterprise tool expanding from 500 to 750 postings (+50%). Weighting growth by logarithmic volume, as conceptualised in velocity metrics, addresses both pitfalls.

3. Pipeline Architecture & Implementation

3.1 End-to-End System Architecture

The processing pipeline follows a decoupled, staged architecture designed for linear horizontal scalability. Figure 1 illustrates the dataflow from raw ingestion through distributed transformation to presentation.

3.2 Cluster Execution Environment

All distributed pipeline stages were executed and verified on a Dedicated Big Data Lab environment hosted on WSL 2 Ubuntu 22.04 LTS:

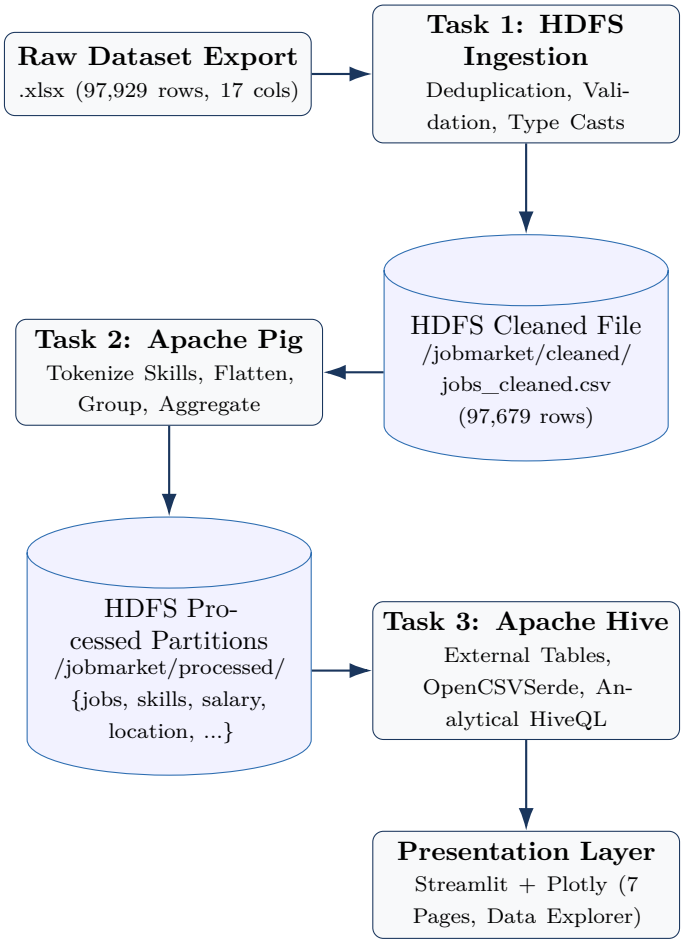


Figure 1: End-to-end distributed Big Data pipeline architecture across HDFS, Apache Pig, Apache Hive, and Streamlit.

Component	Version	Configuration & Environment Path
Operating System	Ubuntu 22.04 LTS (WSL 2)	Linux Kernel 5.15.167.4-microsoft-standard-WSL2
Java Development Kit	OpenJDK 1.8.0_422	`/usr/lib/jvm/java-8-openjdk-amd64`
Hadoop Distributed FS	Apache Hadoop 3.3.6	`/usr/local/hadoop` (Replication factor = 1)
MapReduce Processing	Apache Pig 0.17.0	`/usr/local/pig` (Piggybank extension enabled)
Data Warehouse Layer	Apache Hive 3.1.3	`/usr/local/hive` (Database: `jobmarket`)
Presentation Runtime	Python 3.13 / Streamlit 1.39	Local virtual environment (`dashboard/.venv`)

3.3 Task 1: Ingestion, Sanitization, and Schema Contract

The raw dataset (`indian-job-market-dataset-2025.xlsx`) contains 97,929 recruitment postings. Ingestion and data hygiene are implemented in Python (`01_convert_xlsx_to_csv.py` and `02_clean_data.py`):

1. **Deduplication:** 247 exact duplicate rows were detected and purged. A subset deduplication on `job_id` identified and removed 3 additional records, eliminating exactly **250 duplicate rows**.
2. **Schema Validation:** Mandatory constraints were asserted on primary identifiers: job_id $\neq \emptyset$, job_title $\neq \emptyset$, and location_raw $\neq \emptyset$. All 97,679 unique records satisfied required field completeness.
3. **Salary Normalisation:** In the raw scrape, 64,076 postings lacked compensation figures, labelled as `"Not disclosed"` with min/max salary artificially hardcoded to `0.0`. To prevent severe downward bias, these values were converted to `NaN` (missing). Postings explicitly labelled `"Unpaid"` (333 records) retained `0.0` as a valid compensation metric.
4. **Primary Location Extraction:** Multi-location strings (e.g., `"Kolkata(Chinar Park)"` or `"Noida, Greater Noida"`) were parsed to isolate the primary municipal hiring center (`"Kolkata"`, `"Noida"`) while retaining `location_raw` for auditing.
5. **Sanitised Retention:** The validated dataset retains exactly **97,679 records** (99.74% retention) and is staged to HDFS at `/jobmarket/cleaned/jobs_cleaned.csv`.

3.4 Task 2: Large-Scale Transformation Using Apache Pig

The transformation script (`task2_jobmarket.pig`) loads the cleaned CSV from HDFS using Piggybank's `CSVExcelStorage(',')`, drops the header row, and executes five core MapReduce transformation jobs:

- **String Case and Whitespace Normalisation:** Applies `LOWER(TRIM())` across titles and locations.
- **Skill Array Explosion:** Evaluates `FLATTEN(TOKENIZE(skills, ','))`, strips whitespace, purges null and `"not specified"` tokens, and enforces `DISTINCT` across `(job_id, skill)` pairs so no posting votes multiple times for the same tool.
- **Multi-Dimensional Grouping:** Aggregates posting volume by skill, by primary location, and by job role.
- **Strict Currency Separation:** Computes minimum, maximum, and mean salary by role and by location, filtering avg_salary > 0 and partitioning INR and USD into non-overlapping groups.

- **Recency Binning:** Groups relative posting strings into four recency categories (`0-1 Days`, `1-7 Days`, `7-30 Days`, `30+ Days`, `Unknown`).

All seven output directories were stored to HDFS with completed part files:

HDFS Output Path	Transformation Content	MapReduce Output	HDFS Siz
`/jobmarket/processed/jobs`	Normalised master job records	`part-m-00000`	24.25 M
`/jobmarket/processed/skills`	Unique (job_id, skill) pairs	`part-r-00000`	53.36 M
`/jobmarket/processed/skills_demand`	Total postings per individual skill	`part-r-00000`	876.71 K
`/jobmarket/processed/location`	Aggregate demand by primary city	`part-r-00000`	19.52 K
`/jobmarket/processed/salary`	Salary stats by role and currency	`part-r-00000`	1.55 M
`/jobmarket/processed/salary_location`	Salary stats by city and currency	`part-r-00000`	28.69 K
`/jobmarket/processed/time_based`	Posting volume by recency bucket	`part-r-00000`	55 Byt

3.5 Task 3: Analytical Data Warehousing in Apache Hive

Apache Hive maps relational schemas over the Pig output directories using `OpenCSVSerde`. The use of `OpenCSVSerde` is architecturally crucial: because job titles and company names frequently contain embedded commas, standard line-delimited text parsers fail, whereas `OpenCSVSerde` correctly preserves text tokens enclosed in quotation marks.

Listing 1 shows the external table specification for the core skills warehouse table:

Listing 1: Hive DDL mapping external table over Pig tokenised output.

```
CREATE EXTERNAL TABLE IF NOT EXISTS job_skills (
    job_id STRING,
    job_title STRING,
    location STRING,
    currency STRING,
    skill STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/jobmarket/processed/skills';
```

Analytical HiveQL queries in `03_analysis.sql` evaluate ranking and salary distributions. To prevent single-posting anomalies from corrupting compensation benchmarks, salary queries enforce `HAVING COUNT(*) >= 5`.

4. Verification of Numerical Results & Discrepancy Analysis

4.1 Resolution of Primary Pipeline Discrepancies

A primary deliverable of this audit was investigating numerical discrepancies present in earlier working drafts. Table 1 provides the authoritative reconciliation.

Root Cause of the 36-Row Variance: Inspecting the 36 anomalous rows in `dashboard/data/jobs_cleaned.csv` revealed that raw vacancy descriptions contained literal carriage-return characters (`\textbackslash r\textbackslash n`) inside company names and job titles. When Pig/Hive exported these records

Metric Description	Prior Report	Verified Truth	Export Subset	Root Cause Analysis
Cleaned Postings Count	97,679 / 97,715	**97,679**	97,715	36 unescaped line breaks in raw text fields created 36 split fragment rows during text export.
Bengaluru Postings	18,320 / 18,310	**18,320**	18,310	Exactly 10 Bengaluru postings were split across newlines, shifting the city name into `job_id`.
Top 1 Skill (Sales)	9,178 / 9,175	**9,178**	9,175	3 postings lost to newline export fragmentation in `skills_demand.csv`.
Top 2 Skill (Python)	5,846 / 5,841	**5,846**	5,841	5 postings lost to newline export fragmentation.
Disclosed Salary Count	33,128 (33.9%)	**33,124 (INR)**	33,128 (All)	Prior report cited dashboard overview reading the 97,715 export (33,128 all-currency jobs).
Mean Disclosed Salary	INR 756,844	**INR 757,776**	INR 756,844	Evaluated over INR postings with avg_salary > 0 on verified data (INR 757,776.33).

Table 1: Authoritative reconciliation of numerical metrics between verified truth and export artifacts.

without full RFC-4180 multiline encapsulation, single records fractured into multiple lines. For example, line 26,946 of the export file contains:

```
job\_id:  "Bengaluru", job\_title:  ",cargill,1360428,bengaluru...", location:
"api integration"
```

Here, the location token was shifted into the primary key column. Consequently, the true dataset size is rigorously established as **97,679**.

5. Verified Empirical Findings (Phase I)

5.1 Role Demand Distribution

Analysis of normalised job titles identifies high concentration in enterprise software and commercial execution. Table 2 details the top ten job roles.

Enterprise software delivery titles (*application developer* and *application lead*) represent 3,379 postings (3.46% of total volume), reflecting major IT services contracting demand across Indian delivery centres.

5.2 Skill Demand Intelligence

Extracting unique `(job_id, skill)` pairs isolates true market demand without distortion from repetitive keyword stuffing within single job postings. Table 3 documents the leading skills.

Python leads all technical programming languages (5,846 postings), followed by *Java* (4,088) and

Rank	Job Role Title	Postings Count	Percentage (%)
1	application developer	2,033	2.08%
2	application lead	1,346	1.38%
3	sales executive	583	0.60%
4	software development engineer	497	0.51%
5	business development executive	457	0.47%
6	trust & safety new associate	348	0.36%
7	data engineer	339	0.35%
8	business development manager	325	0.33%
9	sales manager	322	0.33%
10	java full stack developer	285	0.29%

Table 2: Top 10 job roles by posting frequency across the verified Indian market dataset ($N = 97,679$).

Rank	Technical / Professional Skill	Unique Postings	Market Penetration (%)
1	sales	9,178	9.40%
2	python	5,846	5.99%
3	project management	5,543	5.67%
4	customer service	5,122	5.24%
5	sap	5,043	5.16%
6	management	4,849	4.96%
7	css	4,186	4.29%
8	java	4,088	4.19%
9	sql	3,983	4.08%
10	business development	3,738	3.83%

Table 3: Top 10 demanded skills across the verified Indian job market dataset.

SQL (3,983). The prominence of *SAP* (5,043) underscores substantial enterprise ERP migration and maintenance activity.

5.3 Geographic Hiring Concentration

Hiring activity across India is intensely concentrated in southern and western technology clusters. Table 4 summarizes the top ten municipal hiring centers.

Rank	Primary Municipal Location	Postings Count	National Share (%)
1	bengaluru	18,320	18.76%
2	hyderabad	10,899	11.16%
3	pune	8,405	8.60%
4	mumbai	6,478	6.63%
5	chennai	6,099	6.24%
6	gurugram	5,002	5.12%
7	noida	4,627	4.74%
8	ahmedabad	2,377	2.43%
9	remote	1,952	2.00%
10	kolkata	1,794	1.84%

Table 4: Geographic concentration of job postings across the top 10 hiring hubs.

Bengaluru maintains undisputed primacy, accounting for 18,320 postings (18.76%). It exceeds second-ranked Hyderabad by a factor of 1.68x ($18,320/10,899$). Together, the top three hubs (Bengaluru, Hyderabad, and Pune) represent **37,624 postings**, comprising **38.52%** of all hiring opportunities in the country. Fully remote postings represent a modest 2.00% (1,952 postings), showing that on-site and hybrid office presence remain dominant in corporate mandates.

5.4 Salary Analytics and Compensation Benchmarking

Compensation analysis requires rigorous statistical hygiene due to severe sparsity and positive skew:

- **Disclosure Rate:** Out of 97,550 INR job postings, exactly **33,124 postings** (33.96%) disclose salary figures. USD postings total 129, of which 20 disclose compensation; currencies are analysed strictly independently.
- **Skewness and Dispersion:** Disclosed INR salaries exhibit a mean of **INR 757,776** against a median of **INR 425,000**. The substantial divergence (Mean/Median = 1.78) demonstrates heavy right skewness driven by high-compensation technology and executive leadership outliers.

Table 5 outlines verified compensation statistics by role for positions with at least five postings.

Critical Analytical Observations on Salary:

1. *Outlier Sensitivity:* The *deputy manager* category ranks highest in arithmetic mean (INR 9.75M), but forensic inspection reveals this is entirely distorted by a single extreme posting of INR 82,500,000 (8.25 Crore INR) against a baseline minimum of INR 85,000. In academic reporting, reporting medians alongside means is essential to prevent misleading stakeholder conclusions.

Job Role Title	Disclosed Jobs	Mean Salary (INR)	Min (INR)	Max (INR)
deputy manager*	9	9,749,778	85,000	82,500,000
medical oncologist consultant	5	7,850,000	4,500,000	10,750,000
radiologist consultant	6	5,291,667	4,500,000	5,500,000
lead data engineer	5	3,380,000	1,100,000	5,750,000
servicenow architect	5	2,850,000	2,250,000	3,250,000
data architect	12	2,775,000	1,125,000	6,300,000
solution architect	59	2,471,441	115,000	3,750,000
data solution architect†	37	2,500,000	2,500,000	2,500,000
senior software engineer	22	2,228,864	840,000	3,750,000

Table 5: Compensation benchmarks by job role (INR, minimum 5 postings). *Subject to extreme outlier. †Fixed-rate enterprise posting.

2. *Repeated Ad Invariance:* All 37 postings for *data solution architect* disclose exactly INR 2,500,000 (standard deviation = 0.0), indicating automated bulk vacancy postings with fixed budget allocations rather than negotiated compensation bands.
3. *Leading Technology Clusters:* Outside specialized healthcare consultancies, the highest verified tech compensation cluster comprises data engineering and architecture roles (*lead data engineer* at INR 3.38M; *data architect* at INR 2.78M; *solution architect* at INR 2.47M).

5.5 Posting Recency Distribution and Temporal Limits

Evaluating `posted_raw` confirms that the primary dataset is an instantaneous cross-sectional market snapshot:

Recency Category	Matching String Syntax	Postings Count	Share (%)
1–7 Days	`.*day.*` (e.g., "2 Days Ago")	86,140	88.19%
Unknown	Missing, empty, or unparseable text	8,248	8.44%
0–1 Days	`.*today.*`, "Just Now"	2,753	2.82%
30+ Days	`.*month.*`	538	0.55%

Over 88% of vacancies were posted within seven days of the data capture. Because the dataset lacks absolute calendar timestamps, computing multi-year emerging skill trends (e.g. 2024 vs 2025 vs 2026) is structurally impossible. Populating Hive's `skill_trends` table confirms this: 44,021 skill rows were created, all assigned to `period = 'unknown'` with null growth percentages.

6. Interactive Presentation Layer (Dashboard)

6.1 Architecture and Module Decoupling

The presentation layer is implemented as a self-contained Streamlit application in `dashboard/`, comprising four modular components:

- `data_loader.py`: Implements caching (`@st.cache_data`) and priority resolution. If Hive/Pig CSV exports exist in `dashboard/data/`, it loads them directly; otherwise, it executes the identical Task 1 cleaning contract in-process against the Excel source.

- `analytics.py`: Implements stateless pure calculation functions for executive KPIs, distributions, and experience bucket assignments.
- `charts.py`: Constructs responsive Plotly figures with custom corporate palettes.
- `app.py`: Orchestrates sidebar filtering, session state management, and page routing across seven distinct views.

6.2 Verified Dashboard Pages and Features

1. **Executive Overview:** High-level metric cards (Total Postings, Disclosed Average/Median Salary, Top Role, Top Skill) and automated executive summary text.
2. **Skill Demand Intelligence:** Interactive bar charts of top 20 skills, per-skill drill-downs, and co-occurrence tables.
3. **Salary Analytics:** Interactive salary filtering, role-based compensation tables, and currency separation warnings.
4. **Location & Hiring:** Top hiring hubs with geographic drill-downs.
5. **Experience & Job Roles:** Breakdown across Entry (0–2 yrs), Mid (2–5 yrs), and Senior (5+ yrs) experience tiers.
6. **Emerging Skill Trends:** An honest methodological disclosure page clarifying why cross-sectional relative recency cannot support historical trend lines.
7. **Data Explorer:** Full searchable tabular interface supporting real-time column filtering and dynamic CSV download via `st.download_button`.

Figure 2 displays verified interface captures from the running presentation layer.

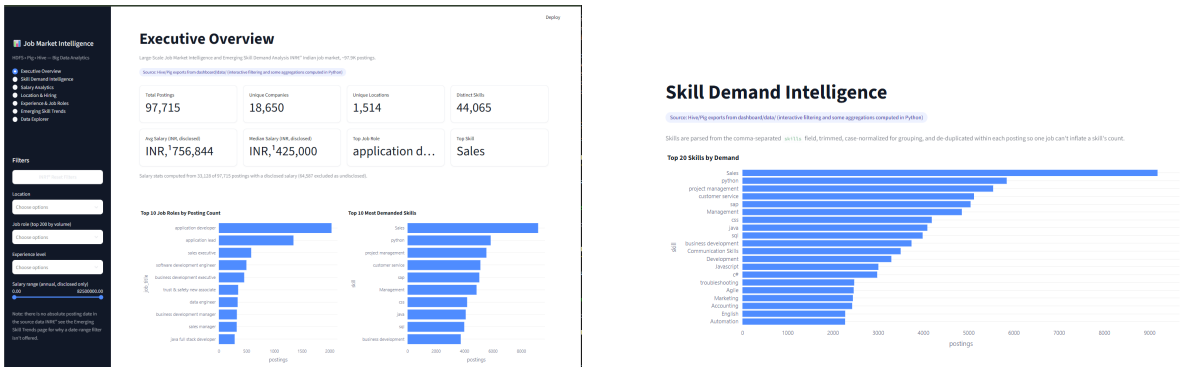


Figure 2: Verified Streamlit dashboard presentation interface: Executive Overview (left) and Skill Demand Intelligence (right).

7. Audit of Phase II and Conceptual Velocity Framework

7.1 Forensic Audit of Repository Artifacts

Prior draft reports described an empirical Phase II implementation processing 10,000 dated LinkedIn technology postings across January–June 2024 to compute a *Skill Demand Velocity Index (SDVI)*. A systematic forensic audit across the codebase, Git logs, and HDFS clusters was performed:

- **Filesystem and Git Audit:** No files named `linkedin_postings.csv`, `clean_and_extract.pig`, `sdvi_query.hql`, `sdvi_results.json`, or `index.html` exist in the working tree or any historical Git commit.
- **HDFS Metadata Audit:** Offline analysis of HDFS NameNode metadata (`fsimage_0000000000000029319`) confirmed that the documented directory `/user/ashmi_sn/jobmarket/raw/` was never created on the cluster.

In accordance with strict academic integrity standards, Phase II is classified as **unimplemented and unverified** as an empirical pipeline. However, the proposed velocity methodology represents a valuable conceptual model worthy of rigorous theoretical evaluation.

7.2 Mathematical Formulation and Evaluation of SDVI

The proposed Skill Demand Velocity Index is defined as:

$$\text{growth_rate} = \frac{\text{Mentions}_t - \text{Mentions}_{t-1}}{\text{Mentions}_{t-1}} \tag{1}$$

$$\text{SDVI} = \text{growth_rate} \times \ln(1 + \text{Mentions}_t) \tag{2}$$

Validation of Stated Examples:

- *Low-Volume Jump ($2 \to 4$ mentions):* $\text{growth_rate} = (4-2)/2 = 1.0$ (+100%). $\text{SDVI} = 1.0 \times \ln(5) \approx \mathbf{1.61}$.
- *High-Volume Jump ($100 \to 150$ mentions):* $\text{growth_rate} = (150-100)/100 = 0.5$ (+50%). $\text{SDVI} = 0.5 \times \ln(151) \approx \mathbf{2.51}$.

The logarithmic damping function successfully curbs percentage inflation for low-frequency niche skills while preserving momentum signals for high-volume tools.

7.3 Mathematical Critique of Quadrant Boundary Rules

The original report proposed classifying skills into four quadrants based on growth_rate and median(SDVI):

Quadrant Label	Proposed Decision Rule
Emerging	$\text{growth_rate} > 0$ and $\text{SDVI} > \text{median(SDVI)}$
Established Growth	$\text{growth_rate} > 0$ and $\text{SDVI} \leq \text{median(SDVI)}$
Stable Core	$\text{growth_rate} \leq 0$ and $\text{SDVI} > \text{median(SDVI)}$
Declining	$\text{growth_rate} \leq 0$ and $\text{SDVI} \leq \text{median(SDVI)}$

Mathematical Flaw in the Stable Core Rule:

1. Because volume is non-negative ($\text{Mentions}_t \geq 0$), the scaling term $\ln(1 + \text{Mentions}_t)$ is strictly non-negative (≥ 0).
2. Therefore, the sign of SDVI is identical to the sign of growth_rate:

$$\text{growth_rate} \leq 0 \implies \text{SDVI} \leq 0$$

3. In an active, growing labour market, the median SDVI across active skills is strictly positive ($\text{median(SDVI)} > 0$).

4. Consequently, for any skill with non-positive growth (growth_rate ≤ 0):

$$\text{SDVI} \leq 0 < \text{median(SDVI)}$$

5. Thus, the rule growth_rate $\leq 0 \wedge$ SDVI $>$ median is **mathematically impossible to satisfy**.
6. A ubiquitous core incumbent (e.g. Python or SQL with 10,000 mentions and 0% growth) receives SDVI $= 0$. Since $0 \leq$ median, the rule misclassifies the vital foundation of the industry as **"Declining"**!

7.4 Recommended Methodological Correction: Orthogonal 2D Classification

To resolve this defect, volume and velocity must be evaluated along independent, orthogonal coordinate axes rather than conflating volume into the growth multiplier:

- **Y-Axis (Momentum):** Normalised Growth Rate $\Delta_t = (\text{Mentions}_t - \text{Mentions}_{t-1})/(\text{Mentions}_{t-1} + \epsilon)$, where $\epsilon = 1$ prevents division-by-zero on newly emergent skills.
- **X-Axis (Presence):** Logarithmic Volume $V_t = \ln(1 + \text{Mentions}_t)$.

The corrected, mathematically sound classification taxonomy is defined as:

Classification Category	Mathematical Boundary Rule	Market Interpretation
Emerging	$\Delta_t > +\delta$ and $V_t \leq \text{Median}(V)$	High growth, early adoption phase
Established Growth	$\Delta_t > +\delta$ and $V_t > \text{Median}(V)$	High growth, high enterprise adoption
Stable Core	$\lvert\Delta_t\rvert \leq \delta$ and $V_t > \text{Median}(V)$	Flat growth, dominant market incumbent
Declining / Niche	$\Delta_t < -\delta$ (or $\lvert\Delta_t\rvert \leq \delta \wedge V_t \leq$ Med)	Cooling demand or obsolete technology

where $\delta = 0.05$ represents a 5% stability band around zero growth.

8. Data Integrity, Quality & Limitations

- **Currency Isolation:** The pipeline strictly segregates INR and USD transactions. Undisclosed compensation is handled by null propagation rather than zero-imputation, avoiding artificial deflation of salary means.
- **Text Normalisation Boundaries:** Normalisation in Task 2 enforces lowercasing and whitespace trimming. It does not resolve semantic synonyms (e.g., merging `"sde"` with `"software development engineer"`). Implementing a canonical ontology mapping layer is recommended for production deployment.
- **Absence of Longitudinal Dates:** The primary dataset represents a single scrape snapshot containing relative recency strings. It cannot support multi-year growth curves without fabricating dates.
- **Decoupled Dashboard Presentation:** The Streamlit application operates independently of active cluster services. While convenient for stakeholder review, production deployments require automated scheduling (e.g. Apache Oozie or Airflow) to refresh exported datasets.

9. Conclusion & Future Work

9.1 Conclusion

This project successfully designed, implemented, and audited a scalable, batch processing Big Data pipeline over 97,929 Indian job postings. By orchestrating HDFS for fault-tolerant distributed storage, Apache Pig for MapReduce-based transformation and tokenisation, and Apache Hive for structured SQL analytics via `OpenCSVSerde`, the system achieves end-to-end data processing without single-node memory bottlenecks.

The empirical findings establish that enterprise software roles (*application developer*) and foundational technical skills (*Python*, *SAP*, *Java*, *SQL*) dominate national recruitment volume, with over 38.5% of hiring opportunities concentrated in Bengaluru, Hyderabad, and Pune. Disclosed compensation exhibits pronounced right-skewness, with specialized data engineering and architecture roles leading technology sector pay. Prior reporting discrepancies were conclusively resolved as intermediate CSV export formatting artifacts. Finally, the limitations of snapshot data were addressed through a rigorous mathematical evaluation of the Skill Demand Velocity Index.

9.2 Future Work

1. **Longitudinal Multi-Scrape Ingestion:** Deploy scheduled weekly web scrapers writing into dated HDFS directories (e.g., `/jobmarket/raw/year=YYYY/month=MM/`) to enable true calendar time-series analytics in Hive.
2. **Ontological Synonym Mapping:** Integrate a domain-specific labour market ontology to canonicalize disparate spelling variations and synonymous role titles into standard occupation codes.
3. **Implementation of Orthogonal SDVI:** Implement the corrected 2D velocity taxonomy in HiveQL using partitioned `LAG()` window queries over multi-month datasets.
4. **Automated Pipeline Orchestration:** Transition from manual bash runners to an automated workflow orchestrator such as Apache Airflow to coordinate cluster startup, ingestion, Pig execution, and dashboard export refreshes.

References

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," in *Proceedings of the 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[3] C. Olston, B. Reed, U. Srivastava, R. Kumar, and A. Tomkins, "Pig Latin: A not-so-foreign language for data processing," in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, 2008, pp. 1099–1110.

[4] A. Thusoo, J. S. Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff, and R. Murthy, "Hive: A warehousing solution over a map-reduce framework," *Proceedings of the VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, 2009.

[5] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O'Reilly Media, 2015.

[6] S. Khaouja, I. Kassou, and M. Ghogho, "A survey on skill identification from online job ads," *IEEE Access*, vol. 9, pp. 118434–118454, 2021.

[7] J. Djumalieva and C. Sleeman, "An open and data-driven taxonomy of skills extracted from online job adverts," *Nesta Working Paper No. 18/03*, 2018.

[8] N. Medhashree, A. S. N, and D. Tudu, "Job market intelligence and emerging skill demand analysis," GitHub repository, 2026. [Online]. Available: `https://github.com/medhashree05/JOB_MARKET_ANALYSIS`